Ep: 3 Your first PHP tag

Keypoints:

① Make website directory called demo [1:51]

② Editor may have some kind of functionality for viewing the results. OR Load it directly on the browser [3:18]

③ To make it dynamic we should swith from HTML to PHP [4:57]

code:

```
<html lang = "en" >
 <head>
      <meta charset = "UTF -8">
      <title> Demo </title>
    </ head>
    <body>
      <h1>
      <?PHP
          echo " Hello World ";
       ?>
      </h1>
</ body>
</ html>
```

~~Q: Why did we a.~~

Q: Why are we told to stick with 'echo' instead of print when we can use ~~both~~ either of them? And what are the ~~sligth~~ differences.

Ep:4 variables:

Key points:

(1) If we want to prepare a string and display it on the page. we can use echo and we place the string within quotes. [1:06]

(2) In PHP we dont use '+', wrong string concatination, in PHP we use the period '.' [2:04]

(3) We create variable using the dollar sigh "$" [3:00]

(4) ~~If~~ If you want to nest and evaluate a variable within a string you have to use double quotes. [7:06]

Scanned with CamScanner

code:

```
<html>
 <head>
   <meta charset="UTF-8">
   <title> Demo </title>
</head>
 <body>
   <h1>
     <?PHP
        $greeting = "Hello";
        echo "$greeting Everybody!"
     ?>
   </h1>
   </body>
   </html>
```

~~Q: can we use numbers as~~

Q: Can we store anything else other than string?

Ep 5 Conditionals and Booleans:

Key points:

(i) Write css [1:07]

(ii) for conditionals we will write using ~~if~~, if, else [5:56] ~~E5:56~~

(iii) Booleans helps the computer answer using true or false [1:15]

Scanned with CamScanner

code:

```
<?PHP
    $name = "Dark matter"
    $read = true;
if ($read){
    $message = "you have read $name.";
}
else {
    $message = "You have not read $name";
}
<h1> <?=message ?> </h1>
```

Q: we have used $read for the true statement, what can we write for false ~~statme~~ statement?

E5 Arrays

Keypoints:

(i) For creating Array we can use [ ]; and store multiple strings [2:40]

(ii) To use trademark symbol at the end we can use the ™ variable enclosed with ( ). [5:31]

(iii) shorthand for loops is a colon [6:48]

code:

```
<?PHP
    $books = [
        "Do Androids Dream of Electric sheep?",
        "The langoliers",
        "Hail Mary"
    ];
?>
<ul> <?PHP foreach ($books as $book): ?>
    <li> <?= $book ?> </li>
    <?PHP endforeach; ?>
</ul>
```

Q: What is the index number in PHP Array?

Ep 7 Assosiative Array:

Keypoints: (i) If you need to put more variable or items with more information inside an array just use another array inside the main array. [3:02]

(ii) Assosiative Array means associating key with a value like a key value pair. [3:40]

(iii) we can create association by an arrow [3:54]

Code:

```
<body>
    <?PHP
        $book = [
          [ 'name' => "Dark Matter',
            'author' => 'Blake Crouch',
            'purchaseUri' => 'http://example.com'
          ]
    ];
?>
    <ul> <?php for each ($books as $book ) : ?>
        <li>
        <a href = "<?. $book['purchaseUri'] ?>">
        <? =$book['name']; ?>
    </a>
</li>
<? php endforeach ; ?>
</UL>
</body>
</html.
```


Scanned with CamScanner

Q: How to get access to multiple links!

Ep 16 Create a MySQL database:

Keypoints:

① We will stick with the defaults because later we will need to reference them our PHP [1:12]

② ~~To save~~ Everything has green background meaning it wasn't saved so we should save it using ctrl + s [3:38]

③ Creating tables inside our database and setting up columns with correct datatypes and primary keys.

code: ~~CREATE~~

```
CREATE DATABASE demo;
USE demo;
    CREATE TABLE posts (
        id INT AUTO-INCREMENT PRIMARY KEY,
        title VARCHAR(255) NOT NULL.
    );
```

Q: What SQL commands do we use to create new database?

Scanned with CamScanner

EP:17 PDO First steps

Keypoints:

(i) we can use select * [1:19]
(ii) To connect PHP to mysql we will use PDO [2:04]
(iii) To initialize PDO, we need to create a new instance of the PDO class [2:34]
(iv) we create a DSN string to specify the database type, host, port and ~~ba~~ database name [5:15]

code:

```
<?PHP
    $dsn = "mysql:host=localhost;port=3306;
    dbname=demo;charset=utf8mb4";
$pdo = new PDO ($dsn, "root", "");
$statement = $pdo->Prepare ("SELECT * FROM posts");
$statement->execute();
$posts = $statement->fetchAll(PDO::FETCH_ASSOC);
foreach ($posts as $post){
        echo "<li>". $post['title']. "</li>";
};
```

Ep-20 SQL injection

Keypoints:

① If we are getting only one record we can stick to fetch [2:56]

② we use $_Get to get the information about the get request [3:60]

③ User table still intact specifically because we leveraged prepared statements with bound parameters [9:23]
- never inline user data into cruise string [9:45]

Code:

```
<?PHP
    require 'functions.php';
    require 'Database.php';
$config = require ('config.php');
$db     = new Database ($config['database']);
$id     = $_GET['id']
$query  = " select * from posts where id = :id "
$posts = $db -> query ($query, [':id' => $id])
                -> fetch
dd ($posts)
```

Q: Why are prepared statements considered much safer against SQL injection than directly embedding variables into your SQL string?

Ep: 25

Keypoints:

(i) Differences between a get request and a post request: Both of them can be used to transfer data or information to the server, but with a get request we are doing it through the query string And with the post request we are doing it sort of in the shadows a little bit as part of the message body [1:08]

(ii) Sticking to a standard list of route names like index, show, create, and store keep our code organized [5:20]

(iii) It's useful if we group them by beginning each controller with the corresponding resource [3:25]

Scanned with CamScanner

code:

```
<?PHP
    $heading = 'Create Note';
if ($server ["Request_METHOD"] = 'Post'
    {
    }
require 'views/note_create.viewPhp';
```

Q: Why do we we the post Method instead of the GET method when submitting a form to create new note in PHP?

EP 26 Always Escape Unfrusted input

(i) Insert note into database [0:33]

(ii) Understanding cross site scripting Risk [2:20]

(iii) Use PHP's built in function to safely convert special characters into safe HTML entities. [8:20]

Scanned with CamScanner

code:

```
if ($server [REQUEST_METHOD ==='Post']
    $db -> query ('INSERT into Notes(body
                                    Users]
        VALUES (:body , :user_id)', [
            'body' => $_POST ['body'],
            'user-id' => 1
        ]);
```

Q: Why do we need input validation if we already using statement and htmlspecialchars() in our PHP programme?

Ep:27 - Intro to form validation

key points:

① Client side vs server side validation [1:32]

② validate input before Database Insertion [2:53]

③ Using (??) to keep previously submitted text in the form [5:39].

code:

```
<? PHP
$config = require ('config.php');
$db = new Database ($config['database']);
$heading = 'Create Note';
if ($server['REQUEST_METHOD'] == 'POST')
{   $errors = [];
    if (strlen ($_POST['body']) == 0),
    > $error['body'] = 'A body is req;'}
    if (strlen ($_POST['body'] > 1000)
    } -$error['body'] = 'The body cannot be
        more than 1000 words
```

```
if (empty ($errors))

    $ db => query ('Insert INTO notes(body,
                    user_id) values ('body',
                                     'user id')
                [ 'body' => $POST ['body'],
                'User_id' => 1
                ]);
    {
}
```

Q: why do we need server side validation even when we have client side validation and how does the '??' operator help ~~prevent~~ preserve a user input.